Reddit-Clone
Microservices

End-to-End Deployment on AWS

PRODUCTION DEPLOYMENT GUIDE

TECH STACK

AWS | Docker | Kubernetes | Terraform

Ansible | Redis | RabbitMQ | S3

A complete, explained, step-by-step guide -
from a fresh AWS account to a running production deployment.
Terraform owns the cloud. Ansible owns the cluster. Kubernetes runs the workload.
Every tool is introduced before it is used. Every phase begins on its own page.

Version 2.0

Reddit-Clone DevOps

AWS - Docker - Kubernetes - Terraform - Ansible - Redis - RabbitMQ - S3

Table of Contents

Part I - Foundations

Before we touch a single command, this part of the guide introduces every tool you will use and shows you how they fit together. You can skim it on the first read, but come back to it whenever something feels unfamiliar later.

In this part

1. Architecture overview

1.1 What you are deploying

Reddit-Clone is a Node.js (TypeScript) microservices backend. It is made up of an API gateway plus eight domain services, talking to PostgreSQL, Redis and RabbitMQ.

Component	Role	Talks to
api-gateway	Routes external HTTP, validates JWTs, rate-limits via Redis	All services, Redis
auth-service	Register, login, refresh tokens	Postgres, RabbitMQ
user-service	User profiles, avatars uploaded to S3	Postgres, RabbitMQ, S3
community-service	Subreddits / communities	Postgres, RabbitMQ
post-service	Posts (text, link, image uploaded to S3)	Postgres, RabbitMQ, S3
comment-service	Comments on posts	Postgres, RabbitMQ
vote-service	Upvote / downvote	Postgres, RabbitMQ
feed-service	Personal and home feed	Postgres, Redis, RabbitMQ
notification-service	User notifications	Postgres, RabbitMQ

1.2 The target AWS architecture

Stateless services run in **EKS**. Stateful infrastructure (DB, cache, broker, object storage) is fully managed by AWS. Everything is provisioned by **Terraform** and wired up by **Ansible**.

From docker-compose	On AWS	Why
postgres	Amazon RDS for PostgreSQL	Managed backups, multi-AZ failover, point-in-time restore.
redis	Amazon ElastiCache for Redis	Same Redis protocol, no servers to babysit.
rabbitmq	Amazon MQ for RabbitMQ	Managed RabbitMQ, AMQPS endpoint.
local volumes / uploads	Amazon S3 bucket	Object storage with lifecycle, encryption, versioning.
Node services	Deployments on Amazon EKS	Stateless, scale horizontally, rolling updates.
Image storage	Amazon ECR	Private registry pulled by EKS nodes with IAM.
External entrypoint	ALB via AWS Load Balancer Controller	TLS, path routing, native to AWS.
Secrets	AWS Secrets Manager + IRSA	No secrets in YAML or images.
Logs and metrics	CloudWatch Container Insights	Logs, metrics, dashboards out of the box.

1.3 Who provisions what

Knowing which tool owns which step is what keeps an IaC project sane. The rule we use is simple: **Terraform owns AWS, Ansible owns the cluster, kubectl owns the application**.

Layer	Owned by	Examples
AWS infrastructure	Terraform	VPC, EKS, RDS, ElastiCache, MQ, S3, ECR, IAM, ACM.
Cluster bootstrap	Ansible	Install Helm charts, addons, ESO, ALB controller, namespaces.
Application	kubectl / CI	Deployments, Services, Ingress, Jobs, HPAs.

2. AWS - the cloud platform

AWS (Amazon Web Services) is the cloud provider that hosts everything in this guide. You rent compute, storage and managed services on demand, billed per second or per request.

2.1 The few AWS concepts you must know

- **Region** - a geographic location like eu-central-1 (Frankfurt) or us-east-1 (Virginia). Each region is independent. Pick one and stay in it.
- **Availability Zone (AZ)** - an isolated datacenter within a region (eu-central-1a, 1b, 1c). For HA we spread across at least two AZs.
- **IAM** - Identity and Access Management. Defines who (a user or a role) can do what (read S3, write to ECR, etc.). Everything in AWS goes through IAM.
- **VPC** - your private virtual network inside AWS. Has public subnets (reachable from the internet) and private subnets (only reachable from inside).
- **Security Group** - a stateful firewall attached to AWS resources. We use these to lock the database down to traffic from EKS nodes only.

2.2 The AWS services we will use

Service	What it is	Used for
EKS	Managed Kubernetes control plane	Running all microservices.
EC2	Virtual machines	Worker nodes for EKS.
VPC	Virtual private network	Network isolation.
RDS	Managed relational database	PostgreSQL for each service.
ElastiCache	Managed in-memory cache	Redis.
Amazon MQ	Managed message broker	RabbitMQ.
S3	Object storage	User uploads, backups, static assets.
ECR	Private container registry	Docker images for each service.
IAM	Access management	Roles, policies, IRSA for pods.
Secrets Manager	Secret store	DB password, JWT secret, RabbitMQ pw.
ACM	Public TLS certificates	Free HTTPS certs for the ALB.
Route 53	DNS	api.yourdomain.com record.
CloudWatch	Logs and metrics	Container Insights, alarms.
ALB (via ELB)	Application Load Balancer	Public HTTPS entry point.

Note: Almost all of those services are provisioned by Terraform in **Phase 4**. You will not click through the AWS Console to create them - you will edit .tf files and run terraform apply.

3. Docker - the container runtime

Docker packages an application together with everything it needs (Node version, system libraries, files) into a single artifact called an **image**. That image runs the same way on your laptop, on a CI runner, and on AWS. There is no "works on my machine" problem when you ship containers.

3.1 The mental model

- **Image** - the immutable filesystem snapshot. Built once with a Dockerfile, never changed.
- **Container** - a running instance of an image. You can have many containers from one image.
- **Registry** - where images live. Docker Hub is public. **Amazon ECR** is our private registry.
- **Layer** - each line in a Dockerfile produces a cached layer. Order them by how often they change (rarely-changing things first) and your builds will be fast.

3.2 What a good Node.js Dockerfile looks like

Every service in this repo ships with its own Dockerfile. They follow the same multi-stage pattern - **build** stage compiles TypeScript, **runtime** stage ships only the compiled output, so the final image stays small (around 150 MB instead of 1 GB).

```
# ---------- build stage ----------
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY tsconfig.json ./
COPY src ./src
RUN npm run build

# ---------- runtime stage ----------
FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=build /app/dist ./dist
USER node
EXPOSE 4001
CMD ["node", "dist/index.js"]
```

3.3 Why we still keep docker-compose.yml

docker-compose.yml is for local development - you spin the whole stack up on your laptop in 60 seconds. In production we use the exact same Docker images, but Kubernetes (not Compose) is what runs them.

4. Kubernetes - orchestration

Kubernetes (k8s) is the system that runs and supervises your containers in production. You declare the desired state in YAML ("I want 3 copies of auth-service running") and k8s makes it true and keeps it true.

4.1 Objects you will meet in this guide

Object	What it does
Namespace	A virtual fence. Everything app-related lives in the reddit-clone namespace.
Pod	The smallest unit of execution. One running container (usually) plus its environment.
Deployment	Tells k8s 'I want N pods of this image, replace them on update, restart them on crash'.
Service	A stable internal DNS name + virtual IP for a set of pods. Internal load balancer.
Ingress	Public HTTP entry. We attach an AWS ALB to it.
ConfigMap	Non-secret env config (hostnames, ports, feature flags).
Secret	Sensitive env (DB password, JWT secret). Synced from AWS Secrets Manager via ESO.
Job	Run-once task. We use Jobs for database migrations.
HorizontalPodAutoscaler	Scales the number of pods based on CPU or custom metrics.
ServiceAccount	Pod identity. We attach AWS IAM roles to ServiceAccounts via IRSA.

4.2 Why we chose EKS over self-managed k8s

- AWS runs the control plane (the etcd cluster, the API server, the scheduler) for us at USD 0.10/hour. We have never had to debug it.
- Tight integration with IAM (IRSA), VPC (CNI), and load balancers (ALB Controller).
- Worker nodes are normal EC2 instances - if we ever need to escape EKS, the workload moves easily.

5. Terraform - infrastructure as code

Terraform is how we describe AWS infrastructure as code. Instead of clicking around the AWS Console, you write .tf files that declare exactly what should exist. Then terraform apply makes AWS look like the code says. Run it again and nothing changes - it is idempotent.

5.1 The two important Terraform concepts

- **State** - Terraform remembers what it created in a .tfstate file. Lose it and Terraform loses track of your infrastructure. We store it in S3 with DynamoDB locking, never on a laptop.
- **Plan vs apply** - terraform plan shows what will change. terraform apply actually changes it. Always read the plan output before applying.

5.2 The Terraform layout we use

```
infra/terraform/
  backend.tf          # S3 + DynamoDB state backend
  versions.tf         # required providers and versions
  variables.tf        # input variables (region, cluster name, ...)
  vpc.tf              # VPC, 3 AZs, public + private subnets
  eks.tf              # EKS cluster + managed node group
  rds.tf              # RDS PostgreSQL
  elasticache.tf      # ElastiCache Redis
  mq.tf               # Amazon MQ RabbitMQ
  s3.tf               # S3 bucket for uploads
  ecr.tf              # 9 ECR repositories
  iam.tf              # IRSA roles
  secrets.tf          # Secrets Manager secrets
  outputs.tf          # endpoints, ARNs, names
```

5.3 Why not the AWS Console?

- Code is reviewable in pull requests.
- Code is reproducible - dev, staging and prod use the same files with different variables.
- Code is destroyable - terraform destroy tears the whole environment down in one command.
- Manual console changes drift away from reality. Terraform tells you when somebody clicked something they should not have.

6. Ansible - configuration as code

Ansible is the second half of the IaC story. Terraform creates the empty EKS cluster, the empty RDS database, the empty S3 bucket. Ansible then walks in and configures them - installs Helm charts, applies Kubernetes addons, seeds initial data.

6.1 Why two tools?

Terraform is great at creating cloud resources but awkward at running commands inside them. Ansible is great at running commands but awkward at creating cloud resources. Together they cover the whole job. The split:

Job	Tool
Create an EKS cluster	Terraform
Install the AWS Load Balancer Controller into that cluster	Ansible
Create an S3 bucket with versioning	Terraform
Create a per-service PostgreSQL database inside the RDS instance	Ansible
Apply Kubernetes manifests	Ansible (or kubectl in CI)

6.2 Concepts you will see

- **Playbook** - a YAML file describing a sequence of tasks.
- **Inventory** - the list of hosts (for us: just localhost, since we orchestrate AWS from our workstation).
- **Role** - a packaged group of tasks (e.g. role: install-helm-charts).
- **Idempotent** - running the playbook twice changes nothing the second time. This is the whole point.

6.3 The directory layout

```
infra/ansible/
  inventory.ini
  ansible.cfg
  bootstrap-cluster.yml     # main playbook
  group_vars/all.yml        # shared variables
  roles/
    helm-charts/            # ESO, ALB controller, metrics server, autoscaler
    eks-addons/             # CoreDNS upgrade, EBS CSI, CloudWatch agent
    rds-databases/          # per-service CREATE DATABASE
    k8s-bootstrap/          # namespace, ConfigMap, ExternalSecret
```

7. Redis - in-memory cache

Redis is a key-value store that lives in RAM. It is the fastest mainstream database you can use - sub-millisecond latency for both reads and writes. We use it for three things in Reddit-Clone:

- **Rate limiting** at the api-gateway. rate-limit-redis counts requests per IP in a sliding window.
- **Feed cache** in feed-service. Hot feeds (top of /r/programming) are cached for 30 seconds so we do not hammer Postgres.
- **Session lookups** in auth-service. Refresh tokens are stored hashed in Redis with a TTL.

7.1 Why ElastiCache instead of a Redis pod?

- Managed backups, automatic failover, no patching for us.
- Multi-AZ - if one AZ goes down the replica is promoted automatically.
- Encryption in transit and at rest are one-tickbox features.
- ElastiCache speaks plain Redis protocol. Our application code does not change at all.

7.2 How the app talks to it

Every service that needs Redis reads three env vars:

```
REDIS_HOST=master.reddit-clone-redis.xxxx.eu-central-1.cache.amazonaws.com
REDIS_PORT=6379
REDIS_PASSWORD=<from AWS Secrets Manager via ESO>
```

ElastiCache requires TLS when transit encryption is on. The Node client opens a TLS socket.

8. RabbitMQ - the message broker

RabbitMQ is how our services talk to each other asynchronously. When a user posts a comment, the comment-service does not directly call notification-service - it publishes a CommentCreated event to RabbitMQ. The notification-service is a subscriber that reacts to that event whenever it can. This decoupling is what makes the system robust.

8.1 The mental model

Concept	What it means
Producer	A service that publishes a message. e.g. comment-service emits CommentCreated.
Exchange	The 'post office' that routes messages. We use one topic exchange reddit.events.
Routing key	A label on a message. e.g. comment.created, vote.cast.
Queue	Where messages land waiting to be processed. Each subscriber has its own queue.
Consumer	A service that reads a queue and processes messages.
Ack / Nack	Confirms (or rejects) that a message was handled. Failed messages can be redelivered.

8.2 Why Amazon MQ?

- Drop-in RabbitMQ - same client libraries, same AMQP protocol.
- Managed brokers, cluster mode for HA, automated patching.
- Private VPC endpoint - the broker is never reachable from the public internet.

Warning: Amazon MQ RabbitMQ requires TLS - use amqps:// on port **5671**, not 5672. This trips people up almost every first deployment.

9. Amazon S3 - object storage

S3 (Simple Storage Service) is AWS's object store. It is effectively infinite, cheap (~USD 0.023 per GB per month) and durable (eleven nines of durability). We use it for three things:

- User-uploaded files - avatars (user-service), post images (post-service).
- Backups of any data we want to keep outside RDS.
- Optional: static frontend assets if you build a web client later.

9.1 Bucket design

- One bucket per environment: reddit-clone-uploads-prod, ...-staging, ...-dev.
- Versioning enabled so an accidental delete is recoverable.
- Server-side encryption (SSE-S3) on by default.
- Block all public access - we never serve from the bucket directly. We hand out short-lived **presigned URLs** to the client.
- Lifecycle rule: move objects to Standard-IA after 30 days, Glacier after 180 days.

9.2 How the app uploads

The pattern is:

- Client asks the backend (POST /uploads/sign) for a presigned URL.
- Backend uses the AWS SDK with credentials from IRSA to generate a 5-minute URL.
- Client PUTs the file directly to that URL - the file never flows through our pods.
- Backend stores the resulting S3 key in Postgres (posts.image_key).

```
import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';

const s3 = new S3Client({ region: process.env.AWS_REGION });

export async function signUpload(key: string, contentType: string) {
  const cmd = new PutObjectCommand({
    Bucket: process.env.S3_BUCKET,
    Key: key,
    ContentType: contentType,
  });
  return getSignedUrl(s3, cmd, { expiresIn: 300 });  // 5 minutes
}
```

The SDK picks up AWS credentials from the pod's service account via IRSA (configured in Phase 10). No access keys live in env vars or in the codebase.

Part II - Hands-on phases

From here on every section is a phase you execute. Each phase begins on its own page so you can print this guide and follow along page by page. Phases are designed to be done in order, but if you redeploy later you can jump straight to the phase you need.

Phases in this part

- Phase 1 - Prepare your AWS account
- Phase 2 - Install local tools
- Phase 3 - Dockerize each microservice
- Phase 4 - Provision AWS with Terraform
- Phase 5 - Configure the cluster with Ansible
- Phase 6 - Push images to ECR
- Phase 7 - Kubernetes manifests
- Phase 8 - Ingress and TLS via ALB
- Phase 9 - Database migrations as Jobs
- Phase 10 - Wire up the S3 bucket
- Phase 11 - Observability
- Phase 12 - Autoscaling
- Phase 13 - CI/CD with GitHub Actions
- Phase 14 - Cost and teardown

Phase 1 - Prepare your AWS account

PHASE 1 of 14 - foundation

Goal: end this phase with an IAM admin user, MFA, billing alerts, and a chosen region. **Time:** ~20 minutes.

1.1 Sign in and pick a region

- Sign in to the AWS Console as the root user at https://console.aws.amazon.com.
- Top-right region selector: pick a region close to your users. Examples: eu-central-1 (Frankfurt), us-east-1 (N. Virginia).
- Write the region code down. You will type it dozens of times.

1.2 Enable MFA on the root user

- Top-right account menu → **Security credentials**.
- Under **Multi-factor authentication** click **Assign MFA device**.
- Use a virtual MFA app (Authy, 1Password, Google Authenticator).

Warning: Never use the root user for daily work. Lock it away after this step.

1.3 Create an IAM admin user

Navigate: **IAM** → **Users** → **Create user**.

- Username: deploy-admin.
- Console access: yes, set a password.
- Permissions: **Attach policies directly** → **AdministratorAccess**.
- Create user. Sign out of root. Sign back in as deploy-admin. Enable MFA on this user too.

1.4 Create programmatic access keys

- As deploy-admin, open **IAM** → **Users** → deploy-admin → **Security credentials**.
- Scroll to **Access keys** → **Create access key** → **CLI** → create.
- Copy **Access key ID** and **Secret access key**. Save them in a password manager - you will only see the secret once.

1.5 Billing alert

- **Billing & Cost Management** → **Billing preferences** → enable billing alerts.
- **CloudWatch** (in us-east-1) → **Alarms** → **Billing** → create alarm at USD 100.
- Send to an SNS topic with your email subscribed.

1.6 Bootstrap Terraform state backend

Terraform state will live in S3 with DynamoDB locks. We have to create that bucket and table by hand once (Terraform cannot create its own backend).

```
aws s3api create-bucket \
  --bucket reddit-clone-tfstate-$ACCOUNT_ID \
  --region eu-central-1 \
  --create-bucket-configuration LocationConstraint=eu-central-1

aws s3api put-bucket-versioning \
  --bucket reddit-clone-tfstate-$ACCOUNT_ID \
  --versioning-configuration Status=Enabled

aws dynamodb create-table \
  --table-name reddit-clone-tflock \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST \
  --region eu-central-1
```

Phase 2 - Install local tools

PHASE 2 of 14 - tooling

Goal: every command in the rest of the guide runs on your workstation. **Time:** ~30 min.

2.1 The toolbelt

Tool	Version	Purpose
aws	v2	Talk to AWS APIs from the shell.
docker	24+	Build and push container images.
kubectl	1.30+	Talk to the Kubernetes API.
terraform	1.7+	Provision AWS infrastructure.
ansible	9+	Bootstrap the cluster, run idempotent config tasks.
helm	3.14+	Install Kubernetes packages.
eksctl	0.180+ (optional)	Convenience tool for IRSA service accounts.

2.2 macOS via Homebrew

```
brew install awscli docker kubectl terraform ansible helm eksctl
```

2.3 Linux

```
# AWS CLI
curl 'https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip' -o awscliv2.zip
unzip awscliv2.zip && sudo ./aws/install

# kubectl
curl -LO 'https://dl.k8s.io/release/v1.30.0/bin/linux/amd64/kubectl'
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# Terraform
wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp
  | sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install -y terraform

# Ansible
sudo apt install -y python3-pip
pip3 install --user ansible kubernetes openshift
ansible-galaxy collection install kubernetes.core community.aws

# Helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

2.4 Configure aws-cli

```
aws configure
# paste your access key id, secret, region (eu-central-1), output (json)

aws sts get-caller-identity   # sanity check
```

Phase 3 - Dockerize each microservice

PHASE 3 of 14 - containerization

Goal: a working Docker image for every service, built locally and smoke-tested with docker-compose.
Time: ~30 min.

3.1 The Dockerfile pattern

Every service folder already contains a multi-stage Dockerfile. The pattern - shown for auth-service - looks like:

```
# Stage 1: install deps + compile TypeScript
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY tsconfig.json ./
COPY src ./src
COPY drizzle ./drizzle
RUN npm run build

# Stage 2: minimal runtime image
FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=build /app/dist ./dist
COPY --from=build /app/drizzle ./drizzle
USER node
EXPOSE 4001
HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget -qO- http://localhost:4001/health || exit 1
CMD ["node", "dist/index.js"]
```

3.2 Smoke-test the whole stack with docker-compose

```
# from project root
cp .env.example .env
docker compose build
docker compose up -d

# in another terminal
curl http://localhost:4000/health
# {"status":"ok"}

docker compose logs -f auth-service
```

Note: If you can hit /health on the gateway and auth-service can read+write to Postgres, you are ready to push images to ECR.

Phase 4 - Provision AWS with Terraform

PHASE 4 of 14 - infrastructure

Goal: one terraform apply creates the VPC, EKS cluster, RDS, ElastiCache (Redis), Amazon MQ (RabbitMQ), the S3 uploads bucket, all ECR repositories, IAM and Secrets Manager. **Time:** ~25 min (most of it spent watching EKS provision).

4.1 backend.tf - remote state

```
terraform {
  backend "s3" {
    bucket         = "reddit-clone-tfstate-<ACCOUNT_ID>"
    key            = "prod/terraform.tfstate"
    region         = "eu-central-1"
    dynamodb_table = "reddit-clone-tflock"
    encrypt        = true
  }
}
```

4.2 versions.tf

```
terraform {
  required_version = ">= 1.7"
  required_providers {
    aws    = { source = "hashicorp/aws",    version = "~> 5.50" }
    random = { source = "hashicorp/random", version = "~> 3.6"  }
  }
}
provider "aws" { region = var.region }
```

4.3 variables.tf

```
variable "region"        { type = string  default = "eu-central-1" }
variable "project"       { type = string  default = "reddit-clone" }
variable "env"           { type = string  default = "prod" }
variable "cluster_name"  { type = string  default = "reddit-clone-prod" }
variable "k8s_version"   { type = string  default = "1.30" }
variable "node_type"     { type = string  default = "t3.large" }
variable "node_count"    { type = number  default = 3 }
variable "services" {
  type    = list(string)
  default = ["api-gateway","auth-service","user-service","community-service",
             "post-service","comment-service","vote-service","feed-service",
             "notification-service"]
}
```

4.4 vpc.tf

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.8"

  name = "${var.project}-${var.env}"
  cidr = "10.0.0.0/16"

  azs             = ["${var.region}a", "${var.region}b", "${var.region}c"]
  public_subnets  = ["10.0.0.0/24", "10.0.1.0/24", "10.0.2.0/24"]
  private_subnets = ["10.0.10.0/24","10.0.11.0/24","10.0.12.0/24"]

  enable_nat_gateway   = true
  single_nat_gateway   = true   # 1 NAT for cost; set false in HA-critical envs
  enable_dns_hostnames = true

  public_subnet_tags  = { "kubernetes.io/role/elb"          = 1 }
  private_subnet_tags = { "kubernetes.io/role/internal-elb" = 1 }
}
```

4.5 eks.tf

```
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "~> 20.13"

  cluster_name    = var.cluster_name
  cluster_version = var.k8s_version
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets

  enable_irsa = true
  cluster_endpoint_public_access = true

  eks_managed_node_groups = {
    default = {
      instance_types = [var.node_type]
      min_size       = var.node_count
      max_size       = var.node_count * 2
      desired_size   = var.node_count
    }
  }

  cluster_addons = {
    vpc-cni                = {}
    coredns                = {}
    kube-proxy             = {}
    aws-ebs-csi-driver     = {}
  }
}
```

4.6 rds.tf

```
resource "aws_security_group" "data" {
  name   = "${var.project}-${var.env}-data"
  vpc_id = module.vpc.vpc_id
}
resource "aws_security_group_rule" "data_from_nodes" {
  for_each                 = toset(["5432","6379","5671"])
  type                     = "ingress"
  from_port                = each.value
  to_port                  = each.value
  protocol                 = "tcp"
  source_security_group_id = module.eks.node_security_group_id
  security_group_id        = aws_security_group.data.id
}

resource "random_password" "db" { length = 32  special = false }

resource "aws_db_subnet_group" "main" {
  name       = "${var.project}-${var.env}-db"
  subnet_ids = module.vpc.private_subnets
}

resource "aws_db_instance" "main" {
  identifier              = "${var.project}-${var.env}"
  engine                  = "postgres"
  engine_version          = "16.3"
  instance_class          = "db.t4g.medium"
  allocated_storage       = 50
  max_allocated_storage   = 200
  storage_encrypted       = true
  username                = "reddit_admin"
  password                = random_password.db.result
  db_subnet_group_name    = aws_db_subnet_group.main.name
  vpc_security_group_ids  = [aws_security_group.data.id]
  multi_az                = true
  backup_retention_period = 7
  skip_final_snapshot     = false
  final_snapshot_identifier = "${var.project}-${var.env}-final"
}
```

4.7 elasticache.tf (Redis)

```
resource "aws_elasticache_subnet_group" "redis" {
  name       = "${var.project}-${var.env}-redis"
  subnet_ids = module.vpc.private_subnets
}

resource "random_password" "redis" { length = 32  special = false }

resource "aws_elasticache_replication_group" "redis" {
  replication_group_id       = "${var.project}-${var.env}-redis"
  description                = "Redis for ${var.project}"
  engine                     = "redis"
  engine_version             = "7.1"
  node_type                  = "cache.t4g.small"
  num_cache_clusters         = 2  # primary + 1 replica
  automatic_failover_enabled = true
  multi_az_enabled           = true
  subnet_group_name          = aws_elasticache_subnet_group.redis.name
  security_group_ids         = [aws_security_group.data.id]
  at_rest_encryption_enabled = true
  transit_encryption_enabled = true
  auth_token                 = random_password.redis.result
}
```

4.8 mq.tf (RabbitMQ)

```
resource "random_password" "mq" { length = 32  special = false }

resource "aws_mq_broker" "rabbit" {
  broker_name        = "${var.project}-${var.env}"
  engine_type        = "RabbitMQ"
  engine_version     = "3.13"
  host_instance_type = "mq.t3.micro"
  deployment_mode    = "CLUSTER_MULTI_AZ"
  publicly_accessible = false
  subnet_ids         = slice(module.vpc.private_subnets, 0, 3)
  security_groups    = [aws_security_group.data.id]
  user {
    username = "reddit_mq"
    password = random_password.mq.result
  }
  logs { general = true }
}
```

4.9 s3.tf

```
resource "aws_s3_bucket" "uploads" {
  bucket = "${var.project}-uploads-${var.env}-${data.aws_caller_identity.me.account_id}"
}
data "aws_caller_identity" "me" {}

resource "aws_s3_bucket_versioning" "uploads" {
  bucket = aws_s3_bucket.uploads.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "uploads" {
  bucket = aws_s3_bucket.uploads.id
  rule { apply_server_side_encryption_by_default { sse_algorithm = "AES256" } }
}

resource "aws_s3_bucket_public_access_block" "uploads" {
  bucket                  = aws_s3_bucket.uploads.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_lifecycle_configuration" "uploads" {
  bucket = aws_s3_bucket.uploads.id
  rule {
    id     = "tier-down"
    status = "Enabled"
    transition { days = 30  storage_class = "STANDARD_IA" }
    transition { days = 180 storage_class = "GLACIER" }
  }
}
```

4.10 ecr.tf - one repo per service

```
resource "aws_ecr_repository" "svc" {
  for_each = toset(var.services)
  name     = "${var.project}/${each.value}"
  image_scanning_configuration { scan_on_push = true }
  image_tag_mutability = "IMMUTABLE"
}
```

4.11 secrets.tf

```
resource "aws_secretsmanager_secret" "db_password" {
  name = "${var.project}/${var.env}/db-password"
}
resource "aws_secretsmanager_secret_version" "db_password" {
  secret_id     = aws_secretsmanager_secret.db_password.id
  secret_string = random_password.db.result
}
# Same pattern for redis-token, rabbitmq-password, and jwt-secret
resource "random_password" "jwt" { length = 64  special = false }
```

4.12 iam.tf - IRSA for app pods (S3 access)

```
module "irsa_uploads" {
  source  = "terraform-aws-modules/iam/aws//modules/iam-role-for-service-accounts-eks"
  version = "~> 5.39"

  role_name = "${var.project}-${var.env}-uploads"
  role_policy_arns = {
    s3 = aws_iam_policy.uploads.arn
  }
  oidc_providers = {
    main = {
      provider_arn               = module.eks.oidc_provider_arn
      namespace_service_accounts = ["reddit-clone:uploads-sa"]
    }
  }
}

resource "aws_iam_policy" "uploads" {
  name   = "${var.project}-${var.env}-uploads"
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = ["s3:PutObject","s3:GetObject","s3:DeleteObject","s3:ListBucket"]
      Resource = [aws_s3_bucket.uploads.arn, "${aws_s3_bucket.uploads.arn}/*"]
    }]
  })
}
```

4.13 outputs.tf

```
output "rds_endpoint"    { value = aws_db_instance.main.address }
output "redis_endpoint"  { value = aws_elasticache_replication_group.redis.primary_endpoint_address }
output "mq_endpoints"    { value = aws_mq_broker.rabbit.instances[*].endpoints }
output "s3_bucket"       { value = aws_s3_bucket.uploads.bucket }
output "ecr_registry"    { value = "${data.aws_caller_identity.me.account_id}.dkr.ecr.${var.region}.ama
output "uploads_role_arn"{ value = module.irsa_uploads.iam_role_arn }
```

4.14 Apply

```
cd infra/terraform
terraform init
terraform plan -out tfplan
terraform apply tfplan

# 20-25 minutes. Most of it is EKS + RDS + MQ creating in parallel.
terraform output
```

Phase 5 - Configure the cluster with Ansible

PHASE 5 of 14 - cluster bootstrap

Goal: the EKS cluster is empty after Terraform. Ansible now installs every piece of cluster-level tooling: External Secrets Operator, AWS Load Balancer Controller, metrics-server, cluster-autoscaler, the reddit-clone namespace, and the per-service Postgres databases. **Time:** ~10 min.

5.1 inventory.ini

```
[local]
localhost ansible_connection=local ansible_python_interpreter=/usr/bin/python3
```

5.2 group_vars/all.yml

```
project: reddit-clone
env: prod
region: eu-central-1
cluster_name: "{{ project }}-{{ env }}"
namespace: "{{ project }}"
services:
  - api-gateway
  - auth-service
  - user-service
  - community-service
  - post-service
  - comment-service
  - vote-service
  - feed-service
  - notification-service
service_dbs:
  auth-service: auth_db
  user-service: user_db
  community-service: community_db
  post-service: post_db
  comment-service: comment_db
  vote-service: vote_db
  feed-service: feed_db
  notification-service: notification_db
```

5.3 bootstrap-cluster.yml - header and helm repos

```
- name: Bootstrap EKS cluster
  hosts: local
  gather_facts: false
  vars_files: [group_vars/all.yml]

  tasks:
    - name: Update kubeconfig
      ansible.builtin.command: >
        aws eks update-kubeconfig
          --name {{ cluster_name }}
          --region {{ region }}
      changed_when: false

    - name: Create application namespace
      kubernetes.core.k8s:
        api_version: v1
        kind: Namespace
        name: "{{ namespace }}"
        state: present

    - name: Add Helm repos
      kubernetes.core.helm_repository:
        name: "{{ item.name }}"
        repo_url: "{{ item.url }}"
      loop:
        - { name: external-secrets, url: "https://charts.external-secrets.io" }
        - { name: eks,              url: "https://aws.github.io/eks-charts" }
        - { name: metrics-server,   url: "https://kubernetes-sigs.github.io/metrics-server/" }
        - { name: autoscaler,       url: "https://kubernetes.github.io/autoscaler" }
```

5.4 bootstrap-cluster.yml - install controllers

```
    - name: Install External Secrets Operator
      kubernetes.core.helm:
        name: external-secrets
        chart_ref: external-secrets/external-secrets
        release_namespace: external-secrets
        create_namespace: true
        values: { installCRDs: true }

    - name: Install AWS Load Balancer Controller
      kubernetes.core.helm:
        name: aws-load-balancer-controller
        chart_ref: eks/aws-load-balancer-controller
        release_namespace: kube-system
        values:
          clusterName: "{{ cluster_name }}"
          serviceAccount:
            create: false
            name: aws-load-balancer-controller

    - name: Install metrics-server
      kubernetes.core.helm:
        name: metrics-server
        chart_ref: metrics-server/metrics-server
        release_namespace: kube-system

    - name: Install cluster-autoscaler
      kubernetes.core.helm:
        name: cluster-autoscaler
        chart_ref: autoscaler/cluster-autoscaler
        release_namespace: kube-system
        values:
          autoDiscovery: { clusterName: "{{ cluster_name }}" }
          awsRegion: "{{ region }}"
```

5.5 bootstrap-cluster.yml - per-service databases

```
    - name: Create per-service databases
      community.postgresql.postgresql_db:
        name: "{{ item.value }}"
        login_host: "{{ rds_endpoint }}"
        login_user: reddit_admin
        login_password: "{{ db_master_password }}"
      loop: "{{ service_dbs | dict2items }}"
```

5.6 Run it

```
cd infra/ansible
ansible-playbook -i inventory.ini bootstrap-cluster.yml \
  -e rds_endpoint=$(cd ../terraform && terraform output -raw rds_endpoint) \
  -e db_master_password=$(aws secretsmanager get-secret-value \
        --secret-id reddit-clone/prod/db-password \
        --query SecretString --output text)
```

Phase 6 - Push images to ECR

PHASE 6 of 14 - images

Goal: every service image lives in ECR with an immutable tag. **Time:** ~15 min the first time.

6.1 Authenticate Docker

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
REGION=eu-central-1
REGISTRY=$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com

aws ecr get-login-password --region $REGION \
  | docker login --username AWS --password-stdin $REGISTRY
```

6.2 Build and push every service

```
TAG=v1.0.0
for svc in api-gateway auth-service user-service community-service \
           post-service comment-service vote-service feed-service notification-service; do
  echo "=== Building $svc ==="
  docker build -t $REGISTRY/reddit-clone/$svc:$TAG ./$svc
  docker push     $REGISTRY/reddit-clone/$svc:$TAG
done
```

Note: Tags are **immutable** in our ECR config. Trying to push the same tag twice will fail - that is intentional. Bump the tag (v1.0.1) for any change.

Phase 7 - Kubernetes manifests

PHASE 7 of 14 - workload

Goal: a ConfigMap, an ExternalSecret, and one Deployment+Service per microservice deployed into the reddit-clone namespace. **Time:** ~15 min.

7.1 ConfigMap (non-secret env)

Save as infra/k8s/configmap.yaml - placeholders are replaced by your terraform output values.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: reddit-clone-config
  namespace: reddit-clone
data:
  AWS_REGION: "eu-central-1"
  POSTGRES_HOST: "<RDS_ENDPOINT>"
  POSTGRES_PORT: "5432"
  POSTGRES_USER: "reddit_admin"
  REDIS_HOST:    "<REDIS_ENDPOINT>"
  REDIS_PORT:    "6379"
  REDIS_TLS:     "true"
  RABBITMQ_URL:  "amqps://reddit_mq:$(RABBITMQ_PASSWORD)@<MQ_HOST>:5671"
  RABBITMQ_EXCHANGE: "reddit.events"
  S3_BUCKET:     "<S3_BUCKET>"
  JWT_EXPIRES_IN: "15m"
  REFRESH_TOKEN_EXPIRES_IN: "7d"
  GATEWAY_PORT: "4000"
  AUTH_SERVICE_URL: "http://auth-service:4001"
  USER_SERVICE_URL: "http://user-service:4002"
  COMMUNITY_SERVICE_URL: "http://community-service:4003"
  POST_SERVICE_URL: "http://post-service:4004"
  COMMENT_SERVICE_URL: "http://comment-service:4005"
  VOTE_SERVICE_URL: "http://vote-service:4006"
  FEED_SERVICE_URL: "http://feed-service:4007"
  NOTIFICATION_SERVICE_URL: "http://notification-service:4008"
```

7.2 ExternalSecret

Save as infra/k8s/external-secret.yaml:

```
apiVersion: external-secrets.io/v1beta1
kind: ClusterSecretStore
metadata: { name: aws-secrets }
spec:
  provider:
    aws:
      service: SecretsManager
      region: eu-central-1
      auth: { jwt: { serviceAccountRef:
          { name: external-secrets-sa, namespace: external-secrets } } }
---
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: reddit-clone-secrets
  namespace: reddit-clone
spec:
  refreshInterval: 1h
  secretStoreRef: { name: aws-secrets, kind: ClusterSecretStore }
  target: { name: reddit-clone-secrets, creationPolicy: Owner }
  data:
    - secretKey: POSTGRES_PASSWORD
      remoteRef: { key: reddit-clone/prod/db-password }
    - secretKey: REDIS_PASSWORD
      remoteRef: { key: reddit-clone/prod/redis-token }
    - secretKey: RABBITMQ_PASSWORD
      remoteRef: { key: reddit-clone/prod/rabbitmq-password }
    - secretKey: JWT_SECRET
      remoteRef: { key: reddit-clone/prod/jwt-secret }
```

7.3 Deployment + Service template

Save as infra/k8s/auth-service.yaml and clone for every other service - change only name, image, port, DB:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
  namespace: reddit-clone
spec:
  replicas: 2
  selector: { matchLabels: { app: auth-service } }
  template:
    metadata: { labels: { app: auth-service } }
    spec:
      containers:
        - name: auth-service
          image: <REGISTRY>/reddit-clone/auth-service:v1.0.0
          ports: [{ containerPort: 4001 }]
          env:
            - { name: AUTH_SERVICE_PORT, value: "4001" }
            - { name: POSTGRES_DB,        value: "auth_db" }
          envFrom:
            - configMapRef: { name: reddit-clone-config }
            - secretRef:    { name: reddit-clone-secrets }
          resources:
            requests: { cpu: 100m, memory: 256Mi }
            limits:   { cpu: 500m, memory: 512Mi }
          readinessProbe: { httpGet: { path: /health, port: 4001 }, periodSeconds: 5 }
          livenessProbe:  { httpGet: { path: /health, port: 4001 }, periodSeconds: 15 }
          securityContext:
            runAsNonRoot: true
            runAsUser: 1000
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: { drop: ["ALL"] }
---
apiVersion: v1
kind: Service
metadata: { name: auth-service, namespace: reddit-clone }
spec:
  selector: { app: auth-service }
  ports: [{ port: 4001, targetPort: 4001 }]
```

7.4 Per-service port and DB map

Service	Container port	DB name
api-gateway	4000	(none)
auth-service	4001	auth_db
user-service	4002	user_db
community-service	4003	community_db
post-service	4004	post_db
comment-service	4005	comment_db
vote-service	4006	vote_db
feed-service	4007	feed_db
notification-service	4008	notification_db

7.5 Apply

```
kubectl apply -f infra/k8s/external-secret.yaml
kubectl apply -f infra/k8s/configmap.yaml
kubectl apply -f infra/k8s/   # everything else
kubectl get pods -n reddit-clone -w
```

Phase 8 - Ingress and TLS via ALB

PHASE 8 of 14 - public entry

Goal: a public HTTPS endpoint https://api.yourdomain.com routed by an AWS ALB to the api-gateway pods. **Time:** ~15 min plus DNS propagation.

8.1 Request the TLS certificate

- **ACM** → **Request certificate** → public.
- Domain: api.yourdomain.com.
- Validation: DNS. Click **Create records in Route 53** if the hosted zone is in this account.
- Wait until status is **Issued**. Copy the certificate ARN.

8.2 Ingress object

Save as infra/k8s/ingress.yaml:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: reddit-clone-ingress
  namespace: reddit-clone
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80},{"HTTPS":443}]'
    alb.ingress.kubernetes.io/ssl-redirect: '443'
    alb.ingress.kubernetes.io/certificate-arn: <ACM_ARN>
    alb.ingress.kubernetes.io/healthcheck-path: /health
spec:
  rules:
    - host: api.yourdomain.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: api-gateway, port: { number: 4000 } }
```

8.3 Point DNS at the ALB

```
kubectl get ingress -n reddit-clone reddit-clone-ingress
# Copy the ADDRESS (k8s-xxx.eu-central-1.elb.amazonaws.com)

# Route 53: create an A-alias record for api.yourdomain.com -> the ALB.
```

After DNS propagation, curl https://api.yourdomain.com/health should return 200.

Phase 9 - Database migrations as Jobs

PHASE 9 of 14 - schema

Goal: every service's Drizzle migrations have been applied to its database. We run them as one-shot Kubernetes Jobs. **Time:** ~5 min.

9.1 Job template

Save as infra/k8s/migrate-auth.yaml and clone per service:

```
apiVersion: batch/v1
kind: Job
metadata: { name: migrate-auth, namespace: reddit-clone }
spec:
  backoffLimit: 1
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: <REGISTRY>/reddit-clone/auth-service:v1.0.0
          command: ["node", "dist/db/run-migrate.js"]
          env:
            - { name: POSTGRES_DB, value: auth_db }
          envFrom:
            - configMapRef: { name: reddit-clone-config }
            - secretRef:    { name: reddit-clone-secrets }
```

9.2 Run them

```
kubectl apply -f infra/k8s/migrate-*.yaml
kubectl get jobs -n reddit-clone -w   # wait for COMPLETIONS 1/1
```

Note: feed-service has no drizzle/ folder in this repo - skip its migration job.

Phase 10 - Wire up the S3 bucket

PHASE 10 of 14 - object storage

Goal: user-service and post-service can write to the S3 bucket without any access keys in env vars, using IRSA. **Time:** ~10 min.

10.1 Create the ServiceAccount with the IRSA role

Save as infra/k8s/uploads-sa.yaml - the role ARN came from Terraform output uploads_role_arn.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: uploads-sa
  namespace: reddit-clone
  annotations:
    eks.amazonaws.com/role-arn: <UPLOADS_ROLE_ARN>
```

10.2 Attach it to the right Deployments

Edit user-service.yaml and post-service.yaml, add:

```
spec:
  template:
    spec:
      serviceAccountName: uploads-sa
```

Re-apply, then restart the pods so they pick up the new identity:

```
kubectl apply -f infra/k8s/uploads-sa.yaml
kubectl apply -f infra/k8s/user-service.yaml
kubectl apply -f infra/k8s/post-service.yaml
kubectl rollout restart deployment/user-service deployment/post-service -n reddit-clone
```

10.3 Verify

```
kubectl exec -n reddit-clone -it deploy/user-service -- sh -c \
  'aws s3 ls s3://$S3_BUCKET --region $AWS_REGION'
# (You will need AWS CLI in the image, or run an aws-cli debug pod with the same SA.)
```

10.4 Smoke test the presigned-URL flow

- curl -X POST https://api.yourdomain.com/uploads/sign -H 'Authorization: Bearer ...'
- Take the returned uploadUrl and PUT a file to it.
- Verify it appears in S3: aws s3 ls s3://reddit-clone-uploads-prod-XXXX/.

Phase 11 - Observability

PHASE 11 of 14 - monitoring

Goal: logs, metrics and alarms for the cluster and the application. **Time:** ~15 min.

11.1 Enable CloudWatch Container Insights

Easiest path: the EKS managed addon.

```
aws eks create-addon \
  --cluster-name reddit-clone-prod \
  --addon-name amazon-cloudwatch-observability
```

Within a minute, Container Insights starts collecting per-pod / per-node CPU, memory, network, and ships container stdout to CloudWatch Logs.

11.2 Where the data lives

- **CloudWatch** → **Container Insights** for dashboards.
- **CloudWatch** → **Log groups** → **/aws/containerinsights/reddit-clone-prod/application** for all pod stdout.
- Logs Insights query example: fields @timestamp, log | filter kubernetes.container_name = 'auth-service' | sort @timestamp desc.

11.3 Alarms worth creating

Alarm	Threshold
ALB 5xx rate	> 1% for 5 minutes
Pod restart count	> 5 in 10 minutes per Deployment
Node CPU	> 80% for 10 minutes
RDS free storage	< 20%
RDS CPU	> 80% for 10 minutes
RabbitMQ message age	> 60 seconds

Phase 12 - Autoscaling

PHASE 12 of 14 - scale

Goal: pods scale up with traffic; nodes scale up if the pods will not fit. **Time:** ~10 min.

12.1 HPA per service

Example for the gateway:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: api-gateway-hpa, namespace: reddit-clone }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-gateway
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 60 }
```

Repeat for every service. metrics-server is already installed by the Ansible playbook in Phase 5.

12.2 Cluster Autoscaler

Cluster Autoscaler is also already installed by Ansible. It watches for Pending pods and grows the EKS node group up to max_size. To verify:

```
kubectl logs -n kube-system deploy/cluster-autoscaler --tail=50
```

Phase 13 - CI/CD with GitHub Actions

PHASE 13 of 14 - automation

Goal: every push to main builds the images and rolls them out. No long-lived AWS keys in GitHub.
Time: ~30 min.

13.1 GitHub OIDC trust

```
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com \
  --thumbprint-list 6938fd4d98bab03faadb97b34396831e3780aea1
```

Create an IAM role github-actions-deploy with a trust policy that allows only your specific repo, and attach a policy with ECR push + EKS describe permissions.

13.2 The workflow

Save as .github/workflows/deploy.yml:

```
name: deploy
on:
  push:
    branches: [main]
permissions:
  id-token: write
  contents: read

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        svc: [api-gateway, auth-service, user-service, community-service,
              post-service, comment-service, vote-service, feed-service,
              notification-service]
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::${{ secrets.AWS_ACCOUNT }}:role/github-actions-deploy
          aws-region: eu-central-1
      - id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2
      - name: Build and push
        run: |
          IMG=${{ steps.login-ecr.outputs.registry }}/reddit-clone/${{ matrix.svc }}:${{ github.sha }}
          docker build -t $IMG ./${{ matrix.svc }}
          docker push $IMG
      - name: Roll deployment
        run: |
          aws eks update-kubeconfig --name reddit-clone-prod --region eu-central-1
          kubectl -n reddit-clone set image \
            deployment/${{ matrix.svc }} \
            ${{ matrix.svc }}=${{ steps.login-ecr.outputs.registry }}/reddit-clone/${{ matrix.svc }}:${
          kubectl -n reddit-clone rollout status deployment/${{ matrix.svc }}
```

Phase 14 - Cost and teardown

PHASE 14 of 14 - shutdown

14.1 Rough monthly cost (eu-central-1, on-demand)

Item	Approx USD / month
EKS control plane	73
3 x t3.large nodes	210
RDS db.t4g.medium Multi-AZ	130
ElastiCache cache.t4g.small + replica	45
Amazon MQ mq.t3.micro cluster	70
ALB	20
NAT gateway (1)	35
S3 storage + requests (small)	5
CloudWatch logs + metrics	20
Data transfer	10
Total (idle)	**~ 620**

Levers to lower it:

- Single-AZ RDS in non-production (about -50% on RDS).
- Spot worker nodes (-50 to -70% on EC2).
- Single-instance MQ in non-production.
- Drop the second Redis node in dev.

14.2 Teardown

Warning: This deletes every byte of data. Take a final RDS snapshot, sync the S3 bucket somewhere safe, then proceed.

```
# 1. Final RDS snapshot
aws rds create-db-snapshot \
  --db-instance-identifier reddit-clone-prod \
  --db-snapshot-identifier reddit-clone-final-$(date +%Y%m%d)

# 2. Sync S3 elsewhere (optional)
aws s3 sync s3://reddit-clone-uploads-prod-XXXX ./backup-uploads/

# 3. Remove app objects from cluster
kubectl delete namespace reddit-clone

# 4. Tear down everything Terraform created
cd infra/terraform
terraform destroy

# 5. Delete the remaining bootstrap items by hand
aws s3 rb s3://reddit-clone-tfstate-$ACCOUNT_ID --force
aws dynamodb delete-table --table-name reddit-clone-tflock
```

Appendix A - Troubleshooting

Pods stuck in ImagePullBackOff

Node IAM role lacks ECR pull permission, or image tag is wrong. Check: kubectl describe pod <name>. Confirm the node role has AmazonEC2ContainerRegistryReadOnly (the EKS Terraform module attaches it by default).

Pods stuck in CrashLoopBackOff

kubectl logs -n reddit-clone <pod> --previous. Usual causes: missing env var (re-check ConfigMap), wrong DB hostname, security group blocking 5432.

ExternalSecret is not creating the Secret

kubectl describe externalsecret -n reddit-clone reddit-clone-secrets. Usual cause: IRSA annotation typo on the ServiceAccount, or the IAM policy does not cover the secret ARN.

ALB Ingress has no ADDRESS after 5 minutes

kubectl logs -n kube-system deploy/aws-load-balancer-controller. Usual cause: subnets are not tagged for ELB. The Terraform VPC module tags them automatically; manual VPCs need kubernetes.io/role/elb=1 on public subnets.

Service can't reach RabbitMQ - ECONNREFUSED 5672

Amazon MQ RabbitMQ uses TLS port **5671** with the amqps:// scheme.

Service can't reach Redis - ECONNREFUSED 6379

ElastiCache requires TLS when transit encryption is on. The Node Redis client needs tls: {} or rediss:// URL scheme.

Service can't write to S3 - AccessDenied

The Deployment must reference the uploads-sa ServiceAccount, AND the pod must be restarted after the annotation was added. kubectl rollout restart the Deployment.

Terraform apply fails with InvalidParameterValue: ELBv2 subnet

VPC subnets must tag kubernetes.io/role/elb=1 (public) and internal-elb=1 (private). Update the VPC config and re-apply.

Ansible Helm task says port forward error

aws eks update-kubeconfig ran for an old user. Re-run it and the kube context will be fresh.

RDS migrations time out from inside the cluster

Confirm the RDS SG allows ingress from the EKS node SG on 5432. The Terraform aws_security_group_rule.data_from_nodes resource handles this.

Appendix B - Quick command index

Inspect the cluster

```
kubectl get nodes
kubectl get pods -A
kubectl get svc -n reddit-clone
```

Tail logs

```
kubectl logs -n reddit-clone -l app=auth-service -f --tail=200
```

Shell into a pod

```
kubectl exec -n reddit-clone -it deploy/auth-service -- sh
```

Roll a deployment

```
kubectl rollout restart deployment/auth-service -n reddit-clone
```

Rollback

```
kubectl rollout undo deployment/auth-service -n reddit-clone
```

Watch HPA

```
kubectl get hpa -n reddit-clone -w
```

Re-run Ansible bootstrap (idempotent)

```
ansible-playbook -i inventory.ini bootstrap-cluster.yml
```

Re-plan Terraform changes

```
terraform plan -out tfplan && terraform apply tfplan
```

Resize node group via Terraform

```
# edit variables.tf: node_count = 5
terraform apply
```

Refresh kubeconfig on a new machine

```
aws eks update-kubeconfig --name reddit-clone-prod --region eu-central-1
```

Force-sync external secrets

```
kubectl annotate externalsecret reddit-clone-secrets \
  -n reddit-clone force-sync=$(date +%s) --overwrite
```

List objects in S3 bucket

```
aws s3 ls s3://$(cd infra/terraform && terraform output -raw s3_bucket)/
```

Get RabbitMQ broker endpoints

```
aws mq describe-broker --broker-id <ID> --query 'BrokerInstances[].Endpoints'
```

Get Redis primary endpoint

```
terraform -chdir=infra/terraform output -raw redis_endpoint
```